HYROS

PRODUCT · API

API Improvement Proposals from Client Needs

Full audit of the public REST API spec v1.37 and the Webhooks spec v1.0, read against real client use cases: automations, CRM and warehouse sync, agency reporting, and compliance.

DATE	PROPOSALS	SPECS AUDITED	METHOD
July 1, 2026	**17 · P0–P2**	**rest-api.txt · webhooks.txt**	**Live-tested in production**

Every claim was tested live against the production API on July 1, 2026 (curl, real account). Endpoints marked 404 were probed with safe bogus ids; write behavior was tested on disposable test leads created for the audit. Each item carries a **Verified** line with the observed result.

The API today is write-heavy and read-poor: clients can push almost any data in (leads, tags, stages, orders, costs, sources), but they can't read much of it back, filter by it, correct it, or delete it. Most proposals below close one of those loops.

ALREADY TICKETED — LISTED FOR COMPLETENESS

- **Ticket 1** — `currentStage` on Get Leads and Get Lead Journey (read parity for stage). *Verified: created a lead with `stage: "test"`; the stage is absent from both the `GET /leads` object and the journey response.*
- **Ticket 2** — `stage` filter on Get Leads. *Verified: `GET /leads?stage=MQL` silently ignores the parameter and returns the unfiltered default list.*
- **Existing card** (per Flavius) — remove tags via API.

P0 Blocking client automations today

1 Remove tags from a lead

Strengthens the existing card with API-level detail.

Today: `POST /leads` and `PUT /leads` only add tags; there is no way to remove one. Tags drive product matching and segmentation, so any client automation built on tag state (add "trial", remove it on purchase) is one-directional. A wrong tag can even generate a false sale (tag matching a product tag creates a sale) with no API path to undo the tag.

> **Verified:** `PUT /leads` with `tags: ["!audit-c"]` on a lead holding `!audit-a, !audit-b` produced `!audit-a, !audit-b, !audit-c` (additive, not replace). `removeTags` and `deleteTags` in the PUT body return `200 OK` but change nothing. `DELETE /leads/tags` and `DELETE /tags/{tag}` return 404.

Proposed fix: add `removeTags: string[]` to `PUT /leads`, or a dedicated `DELETE /api/v1.0/leads/tags?email=...&tags=...`. Symmetric with how tags are added, so integrations need no new concepts.

2 Filter Get Leads by tag

Today: `GET /leads` filters only by ids, emails, and join date. `GET /tags` returns tag names with no lead data. Same intersection problem as the stages tickets: a client can see a tag exists and can tag leads, but can never enumerate "all leads with tag X" to sync a segment into their CRM, email tool, or ad audience.

> **Verified:** `GET /leads?tags="!florida"` (and `tag=`) is silently ignored; the response is the unfiltered default list.

Proposed fix: add `tags` parameter to `GET /leads` (comma-separated, same convention as the proposed `stage` filter). Pairs with ticket 2 so leads become queryable by both classification systems.

3 Incremental sync: `updatedSince` filter on Get Leads

Today: `fromDate` / `toDate` on `GET /leads` filter by **join date** only. A lead created in January and re-tagged, re-staged, or merged in June is invisible to any date-window pull. Clients syncing Hyros into a CRM or data warehouse must re-download the entire lead base to catch updates, which the 30 req/s / 1000 req/min rate limits make impractical for large accounts.

> **Verified:** `GET /leads?updatedFromDate=...` is silently ignored.

Proposed fix: add `updatedFromDate` / `updatedToDate` (or a single `updatedSince`) to `GET /leads`, filtering on last-modified time. Same pattern would help `GET /sales` (refund state changes) and `GET /subscriptions` (status changes).

4 Webhooks: more events, an API to manage them, signed payloads

Today: webhooks exist (separate spec, v1.0) but cover only four events: `sale.attributed`, `lead.opted.in`, `lead.origin.assigned`, `call.attributed`. Three gaps remain:

- **Missing events.** No `sale.refunded`, `lead.stage_changed`, `subscription.created` / `status_changed`, or tag change events. A client can hear about a sale but not its refund, so any revenue mirror drifts; the stage and tag automations from the tickets above have no push path either.
- **No subscription management API.** The webhooks spec documents payloads only; subscriptions are configured in the app. `GET /webhooks`, `/webhooks/subscriptions`, `/events` and similar all return 404 (verified), so agencies provisioning many accounts can't script webhook setup.
- **No payload signing.** The spec defines `eventId` for deduplication but no HMAC signature or shared-secret header, so a receiving endpoint can't verify a payload actually came from Hyros.

Proposed fix: add the missing event types (refund and subscription status first), expose `POST/GET/DELETE /webhook-subscriptions`, and sign payloads with an HMAC header like Stripe's `Signature`. The event infrastructure already exists; this extends it to the cases clients automate around.

5 Strict request validation: reject unknown parameters and fields

New finding from live testing; arguably the most dangerous behavior in the API.

Today: unknown query parameters and unknown body fields are silently ignored. `GET /leads?stage=MQL`, `?tags=...`, `?updatedFromDate=...`, and `GET /sales?orderId=...` all return `200` with the **unfiltered** default list, and `PUT /leads` with `removeTags` returns `200 OK` while doing nothing. A client with a typo (`email=` instead of `emails=`) gets the entire lead base back and thinks the filter worked; an automation "removing" tags believes it succeeded for months.

> **Verified:** every case above reproduced on 2026-07-01.

Proposed fix: return `400` listing the unrecognized parameter/field (the API already does this for missing required params, e.g. journey's "Missing required parameter: ids"). Cheap to implement, prevents silent data corruption in client systems, and makes every future parameter addition discoverable.

P1 Write-only resources clients can't manage

The pattern in this group: the API accepts the data but gives no way to list, correct, or remove it afterward. Every one of these ends in a support ticket today.

6 Products: add GET, PUT, DELETE

Today: `POST /products` only. A client can't list existing products (so scripted creation risks duplicates), can't update a price or `costOfGoods` (which feeds profit and ROAS reporting), and can't remove a product created by mistake.

> **Verified:** `GET /products`, `PUT /products/{id}`, `DELETE /products/{id}` all return 404.

Proposed fix: `GET /products` (paginated, filter by name/tag/category), `PUT /products/{id}`, `DELETE /products/{id}`. The `Product` schema already exists in the spec; this exposes it.

7 Custom costs: add GET, PUT, DELETE

Today: `POST /custom-costs` only. A `DAILY` cost with a typo or no `endDate` skews profit and ROAS on every report from that day forward, and the API offers no way to see it, fix it, or stop it.

> **Verified:** `GET /custom-costs`, `PUT /custom-costs/{id}`, `DELETE /custom-costs/{id}` all return 404.

Proposed fix: `GET /custom-costs` (list with date filters), `PUT /custom-costs/{id}` (edit cost, dates, tags), `DELETE /custom-costs/{id}`. Agencies managing fees across many client accounts are the obvious consumer.

8 Sources: add PUT and DELETE

Today: `GET` and `POST` only. No rename, no changing category/goal/traffic source, no toggling `disregarded` / `organic`, no cleanup of sources created by mistake. Accounts accumulate dead sources that clutter every attribution pull.

> **Verified:** `PUT /sources/{id}` and `DELETE /sources/{id}` return 404.

Proposed fix: `PUT /sources/{id}` for the editable fields and `DELETE /sources/{id}` (or a `disregarded` toggle as the safe minimum).

9 Delete lead (GDPR / CCPA)

Today: no way to delete a lead or its PII via API. Right-to-erasure requests force manual work inside the app, and clients with automated privacy pipelines (common in EU-facing businesses) can't include Hyros in them.

> **Verified:** all five variants return 404, including against a real existing lead id: `DELETE /leads/{id}`, `DELETE /leads?id=`, `DELETE /leads?ids=`, `DELETE /leads?email=`, `DELETE /lead/{id}`. Leads is the only main resource with no delete (sales, calls, and orders all have one). Side effect for this audit: the test leads it created cannot be cleaned up via API.

Proposed fix: `DELETE /api/v1.0/leads?email=...` (or by id), performing the same erasure the app performs. Compliance-driven, so it tends to be a hard requirement in enterprise deals rather than a nice-to-have.

10 Carts: add GET

Today: carts are created and updated via API but only readable through a lead's journey, one lead at a time. The main use case, abandoned-cart recovery ("carts created > 2h ago with no `orderId`"), can't be expressed at all.

> **Verified:** `GET /carts` returns 404.

Proposed fix: `GET /carts` with `fromDate` / `toDate`, `purchased` (has `orderId` or not), and email/leadId filters, paginated. The journey response already defines the cart object shape.

P2 Read-parity and quality-of-life gaps

11 Search leads by phone number

`PUT /leads` accepts `phone` as a lookup key (verified working), but `GET /leads` only accepts ids and emails. Call- and SMS-first businesses (a large share of the call-tracking client base) can't reliably look up the lead a phone number belongs to.

> **Partial undocumented workaround, verified:** leads created without an email get a synthetic address `<phone>@hyrosapi.com`, so `GET /leads?emails=<phone>` finds them via the documented email-prefix search. This only works for phone-only leads: a lead that has a real email is invisible to the same query (verified with a lead holding phone `19995550100`; quoted, unquoted, and `+`-prefixed variants all return empty). `?phones=` / `?phoneNumbers=` are silently ignored and `PUT /leads?email=<phone>` returns 400.

Fix: `phones` parameter on `GET /leads`, same 50-item convention as `emails`, matching the `phoneNumbers` array regardless of whether the lead has an email. Until then, at minimum document the `@hyrosapi.com` synthetic-address behavior, since clients discover it by accident.

12 Lead journey: accept emails, include subscriptions

`GET /leads/journey` requires lead ids, forcing a `GET /leads` round-trip when the client has an email. The journey also returns sales, calls, carts, and linked leads but **not subscriptions**, so the "full journey" is incomplete for any subscription business.

> **Verified:** `?emails=` on journey returns 400 "Missing required parameter: ids"; a live journey response contains no `subscriptions` array.

Fix: accept `emails` as an alternative key; add a `subscriptions` array to the `LeadJourney` schema.

13 Tags endpoint parity: counts, search, pagination

`GET /tags` returns one flat string array: no pagination, no lead counts, no name search. `GET /stages` already does all three (`name`, `amount`, paging), so this is applying an existing internal pattern to tags.

> **Verified:** `GET /tags?pageSize=1` ignores the parameter and returns the full unbounded array. Bonus finding: `GET /stages` returns `"amount": null` for several stages, so even the stage counts need a look.

14 `adOptimizationConsent` read parity

Writable on `POST /leads` and `PUT /leads`, absent from the `Lead` object returned by `GET /leads` and the journey. Same shape as the stage gap in ticket 1: clients set consent but can't audit it, which matters because consent state is itself a compliance artifact.

> **Verified:** created a lead with `adOptimizationConsent: GRANTED`; the field never appears on read.

15 Ad accounts endpoint: document it and let it list

The spec has no way to discover ad accounts, and `GET /attribution` requires ids the client must already know. Live testing found that **`GET /api/v1.0/ad-accounts` already exists but is undocumented**: `?ids=647...` returns `{id, name, type}`. It still requires `ids` ("Missing required field: ids" without them), so it solves nothing for discovery yet.

Fix: document the endpoint and make `ids` optional so it lists all connected ad accounts. That turns it into the bootstrap point every reporting integration needs, probably at the cost of one code path.

16 Async write visibility

Writes are queued: a created lead takes roughly 2 to 4 minutes to become readable, and every write returns only `{"result": "OK", "request_id": ...}`. A `GET` issued right after a successful `POST` returns empty, which integrations read as "creation failed" and retry, creating duplicates. During updates the lead can even briefly disappear from search results.

> **Verified:** lead created at 21:17 became readable at ~21:21; a tag update took ~40s to land and one poll mid-update returned an empty result.

Fix (cheapest first): document the ingestion delay; add `GET /requests/{request_id}` returning pending/processed/failed; long-term, webhooks (proposal 4) make this moot.

17 Consistency and docs-drift cleanup (batch into one card)

Spec vs live API mismatches found while testing, plus spec-internal inconsistencies:

- The live `GET /leads` object includes `firstSource`, `lastSource`, `UTCClickDate`, and an embedded `originLead`; the journey lead adds `isOriginLead`. None are in the documented `Lead` schema. Undocumented fields clients already depend on should be in the spec.
- Phone-only leads get a synthetic `<phone>@hyrosapi.com` email (see proposal 11); undocumented.
- `GET /sales` returns dates as Java `Date.toString()` (`"Thu Jul 02 01:10:33 UTC 2026"`) while every other endpoint uses ISO 8601. **Verified live.**
- `GET /api/v1/domains` is the only endpoint outside `/api/v1.0/`.
- `Source.creationDate` is an integer while every other date in the spec is an ISO 8601 string.
- Stage is written as `stage` (string) on POST but `leadStage: {name, date}` on PUT leads; one shape would simplify client code.
- `GET /sales` offers no `orderId` filter; **verified** that `?orderId=...` is silently ignored (returns other orders' sales), so "give me the sales of order X" requires a client-side scan.

Suggested order

1. Ship the two stage tickets plus tag removal together: they share the theme "make lead classification round-trip" and unblock the most client automations per unit of work.
2. Strict validation (proposal 5) next: one small change that stops every silent-failure class above and makes future filters discoverable.
3. `updatedSince` (proposal 3): it makes every existing GET endpoint useful for sync without new resources.
4. The P1 CRUD group as capacity allows: each one is small, independent, and kills a recurring support-ticket category.
5. Webhooks (proposal 4) as the platform bet: biggest unlock, biggest scope; worth its own design doc.